Testing Strategy

Project Name

CarbonWise

Version: 1.0

Status: Approved

Testing Stack:

- Jest
- Supertest
- Playwright

---

1. Purpose

This document defines the testing strategy for CarbonWise.

The primary objective is to ensure:

- Reliability
- Maintainability
- Correctness
- Security
- Accessibility

while supporting the challenge evaluation criteria.

---

2. Testing Philosophy

CarbonWise follows a testing pyramid approach.

```
                E2E Tests
                    ▲
                    │
          Integration Tests
                    ▲
```

```
                    |
                Unit Tests
```

Testing Priority

Highest Priority

Unit Tests

Reason:

Fast feedback and strong code quality.

Medium Priority

Integration Tests

Reason:

Validate service interactions.

Lower Priority

End-to-End Tests

Reason:

Validate complete user workflows.

---

3. Testing Objectives

The system must verify:

Understand

Users can calculate carbon footprints correctly.

Track

Users can monitor progress accurately.

Reduce

Users receive correct recommendations.

---

4. Test Coverage Goals

Unit Tests

Target Coverage:

```
90%+
```

Integration Tests

Target Coverage:

```
80%+
```

E2E Tests

Target Coverage:

```
Critical User Flows
```

Overall Target

```
85%+
```

---

5. Unit Testing Strategy

Framework

```
Jest
```

Purpose

Validate business logic independently.

Modules Covered

Carbon Calculation Engine

Tests:

- Emission Calculation
- Category Calculation
- Total Carbon Score

Example

```
describe('Carbon Calculator', () => {
  it('calculates car emissions correctly', () => {
    expect(
      calculateEmission(100, 0.192)
    ).toBe(19.2);
  });
});
```

Hotspot Detection Engine

Tests:

- Largest Contributor Detection
- Percentage Calculation

Example

```
expect(
 detectHotspot(data)
).toBe('TRANSPORTATION');
```

Recommendation Engine

Tests:

- Recommendation Ranking
- Priority Calculation
- User Preference Matching

Goal Progress Engine

Tests:

- Progress Percentage
- Milestone Detection

Carbon Twin Engine

Tests:

- Projection Calculations
- Scenario Comparisons

---

6. Integration Testing Strategy

Framework

```
Supertest
```

Purpose

Verify communication between:

- Controllers
- Services
- Database
- Authentication Layer

Areas Tested

Authentication Flow

```
Register

↓

Login

↓

JWT Generation

↓

Protected Route Access
```

Carbon Calculation Flow

```
Submit Activity

↓

Carbon Service

↓

Database

↓

Response
```

Recommendation Flow

```
User Data

↓

Recommendation Engine

↓

Database

↓

API Response
```

Goal Management Flow

```
Create Goal

↓

Store Goal
```

```
↓

Retrieve Goal

↓

Update Goal
```

Example

```
request(app)
  .post('/auth/login')
  .send({
    email,
    password
  })
  .expect(200);
```

---

7. End-to-End Testing Strategy

Framework

```
Playwright
```

Purpose

Validate complete user experiences.

Critical User Flows

Flow 1

User Registration

```
Landing Page

↓

Register

↓

Login

↓

Dashboard
```

Expected Result:

User reaches dashboard.

Flow 2

Carbon Calculation

```
Dashboard

↓

Calculator

↓

Submit Data

↓

Receive Score
```

Expected Result:

Carbon score generated correctly.

Flow 3

Recommendation Journey

```
Dashboard

↓

Recommendations

↓

Accept Recommendation

↓

Create Goal
```

Expected Result:

Recommendation tracked successfully.

Flow 4

Carbon Twin Journey

```
Dashboard

↓

Carbon Twin

↓

Scenario Creation

↓

Forecast Generation
```

Expected Result:

Simulation produced successfully.

Flow 5

Goal Tracking

```
Create Goal

↓

Track Progress

↓

Complete Goal
```

Expected Result:

Goal status updated.

Example Playwright Test

```
test('user login', async ({ page }) => {

 await page.goto('/login');

 await page.fill('#email', 'user@test.com');

 await page.fill('#password', 'password');

 await page.click('button[type=submit]');

 await expect(page).toHaveURL('/dashboard');

});
```

---

8. API Testing

Objectives

Validate:

- Request Validation
- Authentication
- Error Handling
- Response Structure

APIs Covered

Authentication APIs

- Register
- Login
- Refresh Token

Carbon APIs

- Calculate
- Save Activity

Recommendation APIs

- Generate
- Accept
- Complete

Goal APIs

- Create
- Update
- Delete

Carbon Twin APIs

- Simulate

- Retrieve History

---

9. Database Testing

Objectives

Verify:

- Data Integrity
- Constraints
- Relationships
- Indexes

Areas Tested

User Data

Verify:

- Unique Email
- Required Fields

Activities

Verify:

- Valid Relationships
- Correct Storage

Goals

Verify:

- Status Changes
- Progress Updates

Recommendations

Verify:

- Recommendation Tracking
- Recommendation Completion

---

10. Security Testing

Supports:

```
Security (Medium Impact)
```

Authentication Testing

Verify:

- Invalid Credentials
- Expired Tokens
- Unauthorized Access

Authorization Testing

Verify:

- User Isolation
- Protected Endpoints

Input Validation Testing

Verify:

- Invalid Payloads
- Missing Fields
- Incorrect Types

Injection Testing

Verify:

- SQL Injection Resistance
- XSS Protection

Example Payload:

```
<script>alert(1)</script>
```

Expected:

```
Rejected
```

Rate Limiting Testing

Verify:

```
100 Requests / Minute
```

enforcement.

Helmet Verification

Verify security headers:

```
X-Frame-Options

Content-Security-Policy

X-Content-Type-Options
```

---

11. Performance Testing

Supports:

```
Efficiency (Medium Impact)
```

Carbon Calculator

Target:

```
<100 ms
```

Recommendation Engine

Target:

```
<500 ms
```

Dashboard Load

Target:

```
<2 seconds
```

API Response Time

Target:

```
<500 ms
```

Load Testing

Concurrent Users:

```
1000
```

Stress Testing

Concurrent Users:

```
5000
```

---

12. Accessibility Testing

Supports:

```
Accessibility (Low Impact)
```

Standard

```
WCAG 2.2 AA
```

Automated Accessibility Testing

Tools:

```
Playwright

axe-core
```

Manual Accessibility Testing

Verify:

- Keyboard Navigation
- Focus States
- Screen Reader Support

Areas Covered

Forms

Verify:

- Labels
- Error Messages
- Focus Handling

Dashboard

Verify:

- Charts
- Buttons
- Navigation

Calculator

Verify:

- Inputs
- Validation Messages

---

13. Cross-Browser Testing

Browsers:

```
Chrome

Firefox

Edge

Safari
```

---

14. Mobile Testing

Devices:

```
Android

iOS
```

Verify:

- Responsive Layout
- Navigation
- Touch Targets

---

15. CI/CD Testing Pipeline

Every pull request triggers:

```
Code Push

↓

ESLint

↓

Type Check

↓
```

```
Unit Tests

↓

Integration Tests

↓

Build

↓

Playwright E2E

↓

Deployment
```

---

16. Regression Testing

Executed on:

- New Features
- Bug Fixes
- Releases

Purpose:

Ensure existing functionality remains stable.

---

17. Test Data Strategy

Separate environments:

```
Development

Testing

Production
```

Production data must never be used in tests.

Mocking Strategy

Mock:

- External APIs
- Carbon Factor Sources
- Authentication Services

Do Not Mock:

- Core Business Logic
- Recommendation Algorithms
- Carbon Calculations

---

18. Failure Scenarios

The system must handle:

Database Failure

Expected:

```
Graceful Error Response
```

Redis Failure

Expected:

```
Fallback to Database
```

Invalid User Input

Expected:

```
Validation Error
```

Expired Token

Expected:

```
401 Unauthorized
```

---

19. Release Readiness Criteria

A release is approved only when:

✅ Unit Tests Pass

✅ Integration Tests Pass

✅ Playwright Tests Pass

✅ Security Tests Pass

✅ Accessibility Checks Pass

✅ Performance Targets Met

✅ No Critical Bugs Remain

---

20. Success Criteria

The testing strategy is successful when:

✅ Carbon calculations remain accurate

✅ Recommendations remain reliable

✅ Carbon Twin forecasts behave correctly

✅ Security vulnerabilities are minimized

✅ Accessibility requirements are satisfied

✅ Performance remains within targets

✅ Future features can be added confidently

✅ The platform remains maintainable over time